

Final Project

ECE 3710

Group 1011

Henry Wilson, Bobby Lofgren, Kaleb Neilson, Carson Ord

December 8, 2025

Abstract

On Wallstreet, there exist companies that perform super low latency high frequency trading (HFT) to be the first to capitalize on inefficiencies in financial markets. This project explores the replication of these HFT systems. The objective of this project is to use a simulated CPU on the FPGA to interact with market data and make trading decisions. The system continuously receives a small snippet of market data, makes trading decisions with predetermined logic, and executes these decisions on a simulated market. This demonstrates a simplified model of real HFT systems, providing insight into how colossal trading firms operate and make money.

I. Introduction

This project attempts to replicate high frequency trading that is often performed in Wallstreet firms. The primary goal of this project is to have the simulated CPU on the FPGA interact with the internet by making automated trading decisions on a simulated market account. By using embedded Linux on the board's ARM core, two-way communication can be achieved between it and the FPGA utilizing Quartus Platform Designer. The data flow for the project is as follows: C code on the ARM core to extract API calls from Alpaca, filter information into a digestible form for the CPU, send to FPGA for decision making using mmap() function, that decision is then sent back for the C code to send POST request to Alpaca to execute the trade.

II. The CPU

The simulated CPU on the FPGA acts as the decision-making engine. Its purpose is to evaluate snippets of the market and make decisions to buy, hold, or sell. Utilizing the hardware of the FPGA, the data can be analyzed, and these decisions can be made with minimal delay. By using minimal hardware and minimizing trading logic, trades can be executed faster than a standard computer.

The instructions used by the CPU are based on the ISA for a 427 EECS RISC processor. The greatest difference is that the CPU expands the instruction to 32 bits, accommodating the large values used with trading. As the trading logic is quite simple, only some of the 427-processor instructions were used, and even then, not all were needed. The instructions use 24-bit immediate values, split among the upper 16 bits, and lower 8 bits of the instruction. There are no explicit immediate instructions such as "ADDI" for the CPU; the control logic handles whether to use an immediate or register value. All instructions are executed in a fetch-decode-execute three clock cycle.

Instruction format for ALU operations

Bits [31:16]	Bits [15:12]	Bits [11:8]	Bits [7:4]	Bits[3:0]
Upper Immediate 16 bits	0000 if R type Opcode if I type	Rdest	Opcode if R Imm bits if I type*	Imm bits if I Rsrc if R type*

*Shifts operations use the two's complement of the lower 5 bits of immediate or register value, allowing for -16 to +15 bit shifting.

The ALU takes two inputs, A and B, and outputs their result as well as the flags that are set.

Here is the list of instructions and how they work inside of the ALU

AND

- No flags set or used
- Opcode 0001

OR

- No flags set or used
- Opcode 0010

XOR

- No flags set or used
- Opcode 0011

ADD

- Set zero flag if result is 0
- Set overflow flag if the two inputs' MSB are equal, but the result is different
- Set negative flag if the MSB is 1
- Opcode 0101

ADDU

- Set zero flag if result is 0
- Set the carry if result overflows, ie one of the input's MSB is 1, but the result MSB is 0
- Opcode 0110

ADDC

- Set zero flag if result is 0
- Set overflow flag if the two inputs' MSB are equal, but the result is different
- Set the carry if result overflows, ie one of the input's MSB is 1, but the result MSB is 0
- Set negative flag if the MSB is 1
- Uses the carry flag by adding it to B input
- Opcode 0111

LSH

- No flags set or used
- Opcode 1000

SUB

- Set zero flag if result is 0
- Set overflow flag if the two inputs' MSB are equal, but the result is different
- Set the carry if result overflows, ie one of the input's MSB is 1, but the result MSB is 0
- Opcode 1001

SUBC

- Set zero flag if result is 0
- Set overflow flag if the two inputs' MSB are equal, but the result is different
- Set the carry if result overflows, ie one of the input's MSB is 1, but the result MSB is 0
- Uses the carry flag by subtracting it from B input
- Opcode 1010

CMP

- Does not produce a useable output
- Set zero flag if result is 0
- Set negative flag and low flag if MSB of both inputs are the same and $A < B$
- Set negative flag only if MSB of both inputs are different and $A < B$
- Opcode 1011

ASH

- No flags set or used
- Opcode 1101

MUL

- No flags set or used
- Opcode 1110

MOV

- No flags set or used
- Sets output to equal second input
- Opcode 1111

Instruction format for LOAD and STOR instructions

Bits [31:16]	Bits [15:12]	Bits [11:8]	Bits [7:4]	Bits[3:0]
Unused	0100	Rsrc for STOR Rdst for LOAD	0000 for LOAD 0100 for STOR	Raddr

LOAD

- Opcode 0100 in opcode field, 0000 in opcode extension field

STOR

- Opcode 0100 in opcode field, 0100 in opcode extension field

Instruction format for branch and jump instructions

Bits [31:16]	Bits [15:12]	Bits [11:8]	Bits [7:4]	Bits[3:0]
Unused	1100 for Branch 0100 for Jump	Condition	Upper disp for branch 1100 for jump	Raddr for jump Lower disp for branch

Branch & Jump Conditions

- As shown above, bits 11-8 are the conditions in the branch and jump. We didn't implement all of the conditions, but the ones we did implement are listed below. The reason we only implemented signed operations was because our design only dealt with comparing signed numbers. Their mnemonic is "J" or "B" follow by their condition. For example, jump if equal is JE, branch unconditional is BUC.
 - Equal: 0000
 - Not equal: 0001
 - Greater than or equal: 1101
 - Greater than: 0110
 - Less than or equal: 0111
 - Less than: 1100
 - Unconditional: 1110

Lastly, there is a halt instruction made for testing, but since it is not useful for our final program, no mnemonic was made for it. A halt instruction is simply 32'b0.

Components

Our CPU had a lot of different components that we had to design. Putting all of these together was the longest and hardest part of the process. We started in lab 1 with the ALU. In lab 2, we designed the rest of the basic FSM. That came with the reg bank and registers, MUX's, the FSM, and the ALU. For the registers we made 16 32-bit registers that we held in a reg bank. Each register had a reset, enable, in, and out value. The reg bank took a 16-bit number which controlled all of the registers enables. It only took one number at a time, so while we could set multiple registers at the same time if needed, they had to be the same number. Two of the MUX's were 16-to-1. We put in all of the registers from the reg bank, and the control was from the FSM which selected which registers we wanted to rSrc and rDst. We also added a 2-to-1 MUX. This MUX took the output from the rDst MUX and an immediate. This was our

immediate select MUX, so that we could implement immediate operations. Our FSM went through states and operations, while setting outputs that control everything in our data path. It had lots of outputs that were connected to the reg bank, MUX's, ALU, etc. We implemented multiple FSMs for this lab, but the one we demonstrated was the Fibonacci. In every state, we took the previous two registers and added them for the next register. Our data path for lab 2 is shown in Figure 1.

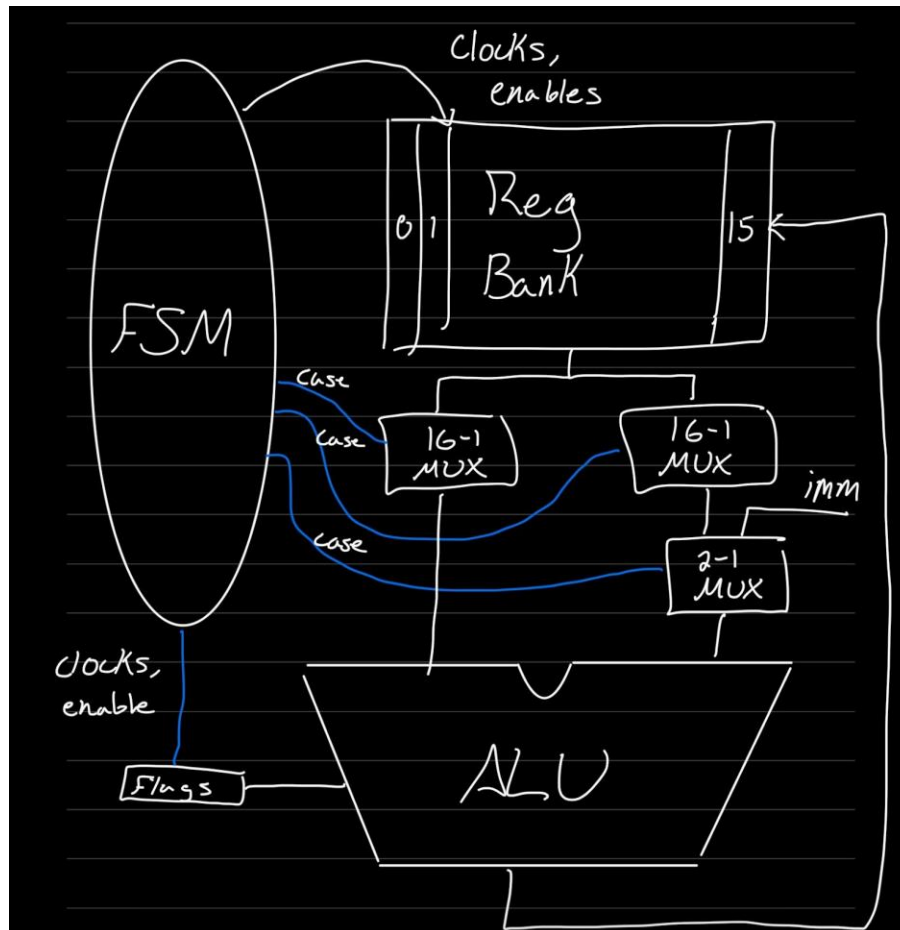


Figure 1. Lab 2 Datapath

In lab 3, we implemented our memory. We implemented a dual port-based memory that took in two inputs, enables, addresses, and had two outputs. We had to make sure the addresses were never the same at the same time, otherwise we could lose the data that we were trying to write. If the enable was 0, then we wanted to read from the input address, while if the enable was 1, we wanted to write the data that we input to the address we input. To demonstrate this lab work, we wrote a basic FSM that just talked to memory with new data every cycle. The final lab we had to implement to finish our CPU was lab 4. In lab 4, we had to put everything together needed for our final CPU. We had to add an encoder, PC, FR, displacer, IR, more MUX's, and a final FSM. Designing everything on its own was the simpler task. The PC is just a register that holds where we are in our instruction set. The displacer was just an adder that adds a different

value based on the control input. The IR is another register that holds the current instruction. Designing the FSM and tying everything together was one of the most pscp pscp-test.txt paul@192.168.1.23:/home/paul nging aspects of the entire project. As suggested, we implemented our CPU incrementally. We started by designing the FSM to only handle R/I type instructions. Our FSM followed a simple structure. It would fetch the instruction, decode the instruction, and perform actions in a specific state based on the instruction. Of lab 4, this took us the longest, since getting it to work once was one of the most challenging tasks. We got it to work for R/I instructions while setting and enabling everything we needed in the FSM. Once that was working, we moved on to loads and stores. Here, we had to combine the memory we got working in the last lab. These instructions took a bit more logic, since we had to access values in memory or set them. The final instructions we needed to implement were jump and branch. All we had to add to accomplish this was check our flags register for the condition code, and then set the displacement based upon the instruction. Our final data path for our CPU is shown in Figure 2.

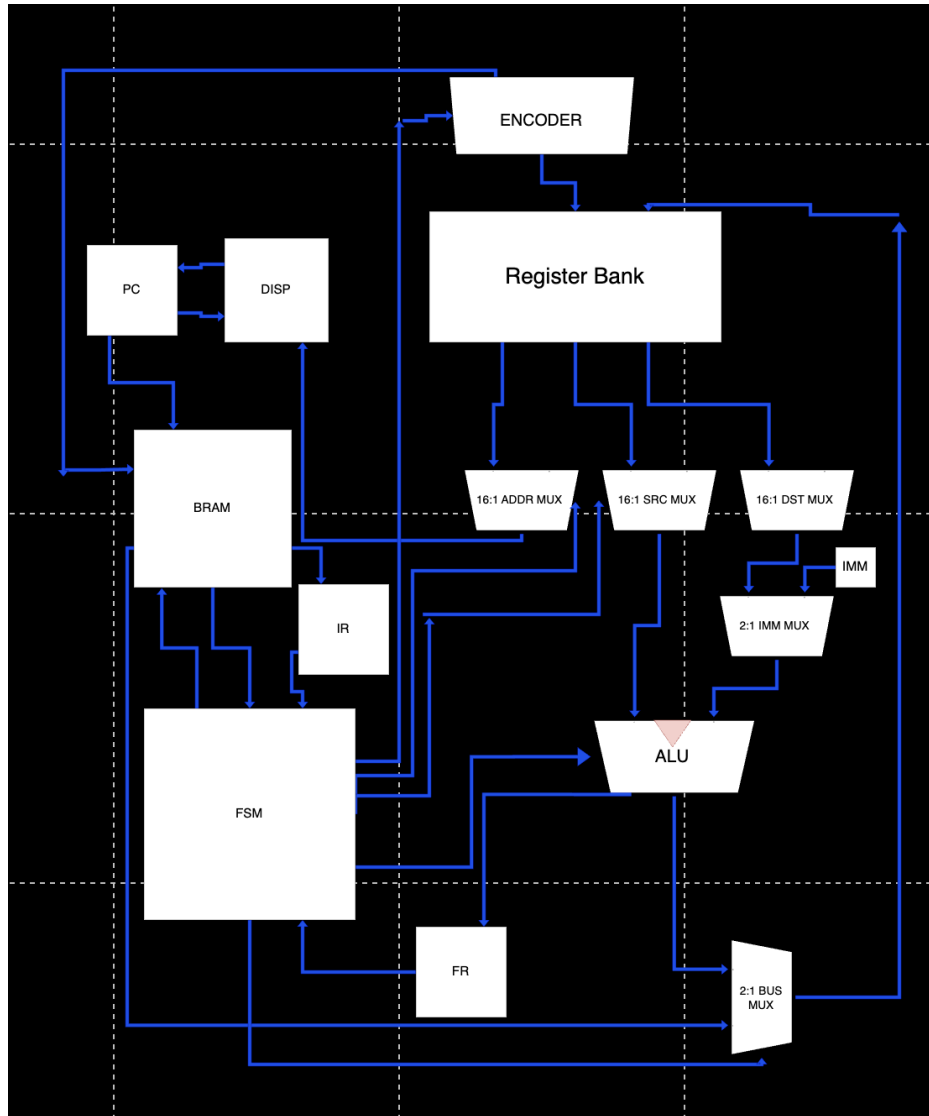


Figure 2. Final Datapath

III. Peripherals

The purpose of the system is to perform automated trades quickly, so no peripherals were used for user input or display. The only true peripheral used was the ARM core HPS that is on the same board as the FPGA. The HPS allows us to use software that can communicate with both the website and the FPGA CPU. Utilizing the AXI protocol, we were able to create a bridge between the FPGA BRAM and the HPS, allowing for two-way communication. Since trades will be executed on a simulated market already, we utilize the display already built into the website dashboard instead of a VGA display. A short description of each component of this project is shown in figure 3.

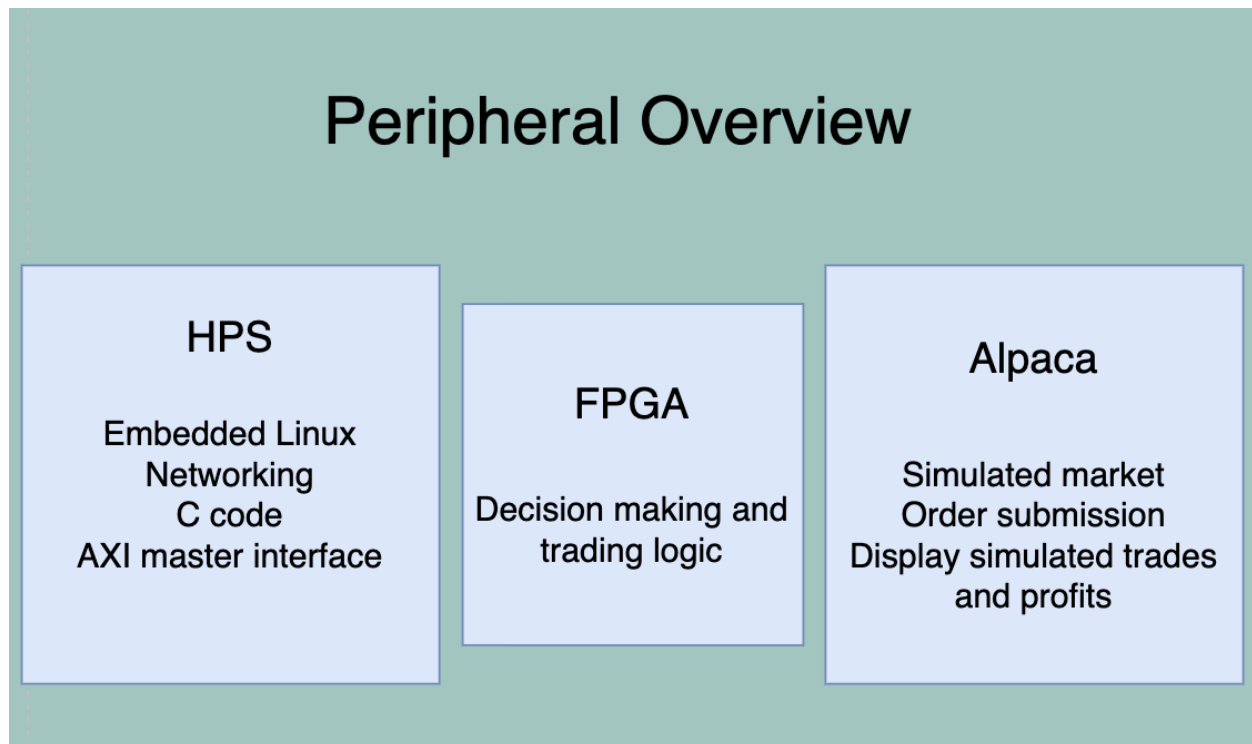


Figure 3. Overview of Peripherals

The HPS is essential for the success of this project because it provides all of the capabilities that would not be feasible to implement on the FPGA alone, at least not in the scope of this class. It runs embedded Linux, handles the TCP/IP, JSON parsing, WebSocket interface, and API authentication. Offloading all of the high-level work to the HPS has made this project possible, while still implementing the FPGA for quick decision making, which is its primary purpose in cutting edge HFT systems.

IV. Software

We had to implement multiple different software applications to make our integration work. We implemented an assembler, our main program with our trading logic, and a program that reads and writes information from trading software.

The first program we wrote was our assembler. We wrote this, so we could turn normal assembly code into our 32-bit instructions. Our assembler was a basic assembler written in Python. We used this assembler as a tool to make writing machine code easier. Our assembler was very important, because without it we would have had to write our own 32-bit instructions. We tested this assembler thoroughly, since even though we followed the ISA for our instructions, if anything was off, our program wouldn't work. We tested all different types of instructions and codes in our assembler to make sure it would produce the correct 32-bit instructions. We began

this process immediately after finishing lab 4, but in hindsight we would have started it earlier, since we needed lots of machine code to test our CPU.

The next two programs worked together but could be written and tested on their own. The machine code we needed to execute our logic was written next. There were a lot of decisions that went into our logic. We wanted to design an effective trader that makes good and fast decisions. Due to some challenges, mostly out of our control, that will be mentioned in section VI, that was not possible. We then had to decide that we wanted our program to execute trades, even if they weren't the most possible. Most trading algorithms are quantum algorithms that we could not have written in our time in this class. Upon further discussion we decided to stay with our original algorithm, but we would simplify it. We wanted to check one stock at a time for its buy and sell price. Occasionally, the sell price will be higher than the buy and when our program notices that it signals back to the trading program to buy and sell simultaneously. Our machine program was a loop that ran forever when we wanted it to. At the top of the program, it loads in the most recent buy and sell prices from memory. It compares the two prices to see if the last sell is higher than the last buy. If it is, we set the last register to 1, signaling the trader to make the transaction. If not, we skip the last step and run the program over again. Sometimes there will be new data to be read in memory, and sometimes there will not be, depending on the speed of the Linux.

The other program we had to write was the trader program. This program reads data from a stock trading website and outputs it to Bram on the FPGA. It also took in data, so that if we wanted to buy/sell stocks, it would send that to the trading application. We wrote this program in C, since it would be executed on Linux on the board, and C is the only language it could support. This program took a lot of trial and error, since a lot of trading applications aren't free, don't work in Utah, or don't have the information we need. We ended up using the trading website Alpaca. We were able to read the information we wanted from it, and then the program would parse it into the information we wanted to send to the FPGA. It also took input, so that if we wanted to make a transaction, it would order the information and send it to the website.

These two programs needed to work together for our design to work. The trading program needed to parse the information necessary for the machine code. It had to set the correct things in Bram, so that the machine code could access it and make decisions. The machine code had to output the right data, so that the trading program can make and send decisions back to the application. We wrote and tested these both on their own, and then once they were working, we implemented them together.

V. Overall Integration

The project integrates the HPS and the FPGA, allowing them to work together for communication with the websites and executing trade decisions. The ARM core HPS running Linux handles communication with Alpaca and Alpaca. By using the `mmap()` function, incoming market data from Alpaca can be written directly to the FPGA hardware registers. The Platform Designer allows for a bus protocol to share the same BRAM location between the HPS and FPGA, so both systems can read and write to the same locations. This allows the HPS to write market data to BRAM, so the FPGA can read and write trading decisions back for the HPS to read.

With this design, the HPS can act as a bridge between the internet and the FPGA. We were able to parse information into a digestible form for the simulated CPU to use. When the HPS detects a signal in shared BRAM for execution of a trade, the software sends a POST request to the Alpaca simulated market to submit that order.

While Figure 4. shows top to bottom architecture, it is more of a loop. Every cycle the HPS gets new market data and writes that new data to the FPGA. The FPGA constantly runs over the same program. If the data is the same, then the output from the FPGA will not change, but when the FPGA gets new data, it will execute the algorithm on the new set and produce a new output. The HPS is constantly reading the output to send to the trading program.

Our integration had three main parts. Getting all of these parts to work individually did not take a lot of time, except for the HPS to FPGA communication. The hardest part was getting all of them to work together. Our three parts were the HPS to FPGA communication, the trading program that talked to the application and parsed data, and our CPU with our trading algorithm. When we got all of these parts working, we started to put them together but ran into some issues. Our HPS only supported C, but our trading program was written in Python. It took some debugging, but we eventually got our trading program to work in C. Getting our integration working was challenging, but at the end we had everything working together.

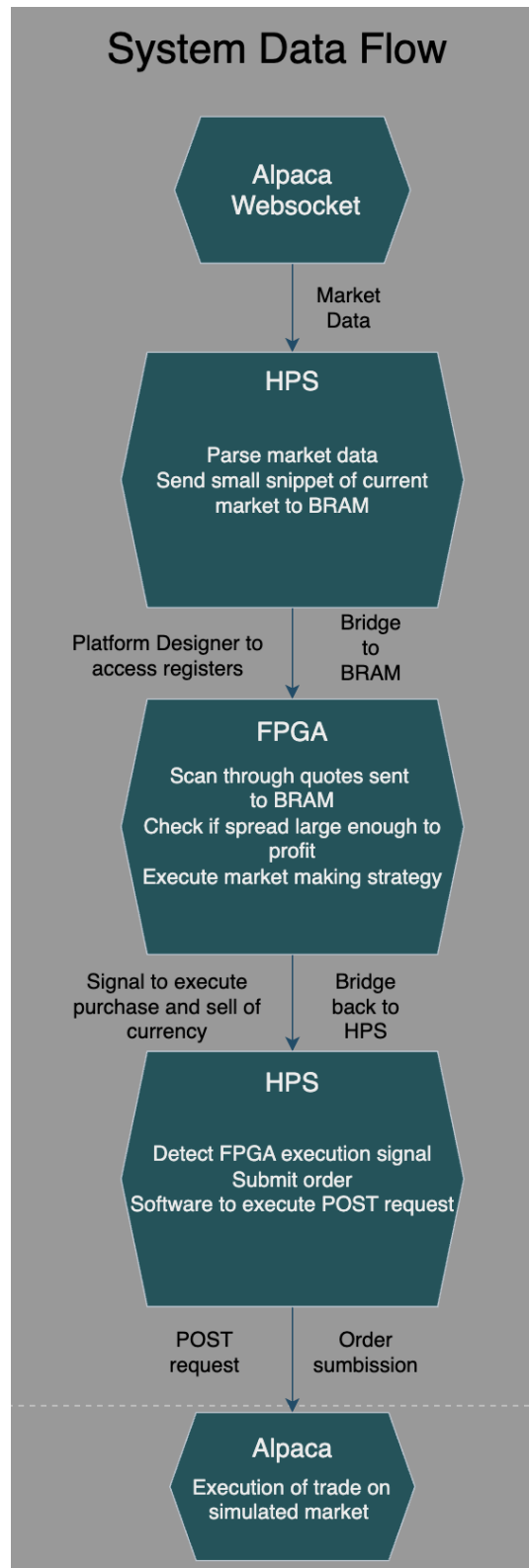


Figure 4. System Data Flow

VI. What We Learned

We learned a lot over the course of this semester, not only about topics in this course, but also how to work better as a team and manage things better. In terms of topics in this course, we learned a ton. None of us knew anything about the HPS on the FPGA and its communication. We had to teach ourselves all of it, and while it took a lot of trial and error, we got it working in the end. This is where most of our technical issues came from. We had no idea how any of it worked, so we had to try to learn it and apply it in one semester. We had no other peripherals, so we needed this to work for a complete project. There were a lot of different ideas going around for how we would get this to work, and we eventually had to try a lot of different things. We started with the idea of using an Ethernet cable to support connection and communication in these two ports. This would have taken a lot of extra work and code, as we would have needed drivers and other things for this communication to work. We wanted to use Ethernet because it was fast, and we wanted our project to be fast to actually make successful trades. We realized too late that this was not going to work, and we had to pivot it. We decided to use the `mmap()` function to the FPGA from HPS to do the communication. This ended up being a lot simpler, and even though our communication wasn't as fast as we wanted, it was still fast, and it worked. That is the biggest lesson we learned during this course. We had super high expectations at the beginning, but over the course of the semester, we realized that a lot of our ideas weren't possible in a few short months. We understood that a working product was better than nothing, and that is why at the end we worked really hard to get our project working.

We also made a few mistakes in the design of labs 3 and 4. When we designed lab 3, our testbench was not correct, and while it displayed correct results, we were not actually doing it correctly. Luckily, we noticed quickly and since lab 3 was not the biggest lab, we were able to quickly rectify the problem. In lab 4, we almost got to the end of the lab, but our code ended up not working. We had to rewrite all of lab 4 from scratch in one week to meet the deadline. We ended up making a few small code errors in the first part of lab 4, causing none of the instruction sets to work. We worked really hard as a team to fix this by the deadline, and we were able to fix lab 4 with our specifications. That part taught us that we needed to test thoroughly and make sure our test benches were correct.

The HPS integration was a particularly challenging section of this project. As it was beyond the scope of this class, there was a lot to learn. It required knowledge in digital design, networking, Linux, C programming, and a lot of research. First, we needed to get embedded Linux installed onto the HPS. We followed the steps listed on Cornell's ECE website [1]. We were able to load this onto a microSD card and insert it in the DE1-SoC board. Once this was done, we needed a way to communicate. PuTTY was the software of choice for an SSH connection to the embedded Linux. It required a serial connection under the appropriate COM port and a serial of 115200.

Now that we have a way to communicate with Linux, we must set it up for internet connection as this does not come by default. First, we need to add the Google DNS IP to our DNS list. This is done by the `nano /etc/resolv.conf` command. Here, we add `nameserver 8.8.8.8` and `nameserver 8.8.4.4`. Next, we need to set the default gateway for Linux. This requires use of

the following command: `ip route add default via <your gateway>(eg:192.168.1.1) dev eth0` [2]. The gateway is the IP address of your router (this cannot be done on school grounds without an IP/Mac exception, we had to do at home). The final networking step is to set up file sharing between your personal computer and linux (this can be avoided if you are smart and know how to use git). This requires the installation of the PSCP (PuTTY Secure Copy) software on your machine [3]. Once this is done, and assuming your Linux and PC are on the same network, you can use the command “`pscp pscp-test.txt paul@192.168.1.23:/home/paul`” where you choose what file to move over and the address of your Linux (was `root@192.168.1.123` for us).

Now we need to get the embedded Linux on the HPS talking with our FPGA. This part required a LOT of research as there were no fully documented cases for this type of setup. We chose to use C language for its fast `mmap()` function. With `mmap`, we can write directly to a BRAM address on our FPGA. You first must install C on the Linux machine with the command “`apt-get update`”, followed by “`apt-get install build-essential`”. Next, we need to build a physical link between the HPS and our system. This requires the use of the Platform Designer software that is built into Linux. From here, we can instantiate an On-Chip RAM module (this eventually replaced our initial BRAM), as well as the Arria V/Cyclone V Hard Processor System module. Once these have been added, we followed the steps of a YouTube video to get the general understanding of how the connection works [4]. Personally, the compilation of the .qsys file from Platform Designer required the installation of a Linux subsystem on my windows machine. Following the steps from Intel’s website, we can get this installed [5]. Now that we’ve got a dual-port single clock BRAM module created, we can export the s2 port for the BRAM to be used in our verilog, with s1 being connected to our HPS over an AXI Master-Slave connection. It is important to note that addresses chosen in Platform Designer for your BRAM are not the same address names you will write to on the Linux system. For the AXI Master bridge, the Linux `mmap` base address will be `0xC8000000` [6]. If you decide to use the slower Lightweight AXI Master, the address is `0xFF200000`. Our C code shows how to properly use these addresses.

Finally, with everything configured for connection, we can write the C code that pulls market data from the internet (we chose the Coinbase WebSocket as it gave us the ability to pull data once every 200ms) and input it directly into the FPGA for calculation. Ideally, this is done with an ethernet stack decoding messages directly sent from a stock exchange; however, we do not have access to any stock exchange data cables in SLC. This was the lowest latency solution we could use given our location. To execute the trades, we planned to use the Alpaca trading API as it allows us to create a “paper trading” account and test our infrastructure. We were unable to get our FPGA to properly read the information sent from the HPS and have it calculated properly due to time constraints. If we had more time, we would be able to get our data written to BRAM appropriately, as well as have a program written to correctly read the information provided by the HPS. If we were able to sort that out, the Alpaca integration would be the next step, and we would be on our way to implementing a full-stack HFT Market Making Scheme with profit.

This course taught us a lot about digital design and the process of designing a project as a team. Even though we had our problems and made multiple mistakes, we all worked very hard together to produce a working final product.

VII. Group Responsibilities

Each person in our group had their own assigned responsibilities. Everyone was able to get their part of the project done, and without everyone working together we wouldn't have had a complete product. There were parts that everyone did together, like the design of earlier labs. We worked very well as a team, and being able to split up the work the way we did made everything possible.

Kaleb did all the HPS to FPGA communication. He took the task of getting the HPS on the board to talk to the rest of it, allowing our program to be able to work. This was absolutely crucial because without this, none of our project would have worked. Once he got communication working, he took programs written by Bobby and Carson to communicate the information we wanted in our program. He also added everything to the imbedded Linux on the board that was necessary for our project.

Henry designed our CPU for lab 4 and wrote the assembler. We all worked together on lab 4, but Henry did the main parts of designing and integration. He designed our data path, so that it would work, and he also wrote a lot of the smaller files needed in the CPU. He also wrote the assembler. This was needed to turn all our assembly code into machine code, and he designed and tested it for our specific 32-bit instructions. He designed it so that it would work with all the instructions needed based on our ALU and the ISA.

Bobby did all of the testing for the early labs, along with Carson. He wrote the test benches and did necessary debugging to make the labs officially work. He made sure the earlier labs worked before we moved on to the next parts. He also wrote the assembly used in our final program. He wrote and tested this program, which made trades based on data from memory. He also wrote lab reports. He compiled data and was able to write reports for the earlier labs.

Carson also helped with testing labs. He also wrote test benches and debugged labs that did not work. He also wrote the code that talked to the trading program. He wrote all of the code that talked to the trading program. He parsed the information we needed in the way that we needed it. He also wrote code to take in information from the FPGA, to be sent to the trading program to execute trades. He wrote and tested all this code to ensure that it worked the way we needed it.

VIII. Conclusion

This project has met our original goal of using an FPGA to execute a market-making strategy. We were able to utilize the HPS in a system that can take advantage of the low latency FPGA hardware. Live market data was collected on the HPS, sent to the FPGA over AXI protocol, processed on the simulated CPU in the FPGA, and a signal was sent back to the HPS to deliver to Alpaca for trade execution. While we were unable to reach our stretch goal of simulating enough profits to trust the system with real money, it serves as a proof of concept for a more fleshed out high frequency trading system.

The project was relatively bare bones and slow compared to systems in high frequency trading firms. In the best case, we could achieve 5 API calls per second from a data source that is likely slower to update than a physical exchange. We also heavily relied on the HPS and higher-level software that increases the latency. Another is for improvement was that the scope of currencies used was narrow. Since the S&P 500 is only open for limited hours, we opted to trade cryptocurrencies that are open 24/7 for accessibility.

In the future, this project can be expanded to be faster and more capable. The greatest bottleneck is the lack of a direct connection. Since we are limited to several API calls per second from a website, we have two options to improve this. We could bring the system to a physical exchange, but this often requires a hefty payment to access. A more feasible approach would be to trade obscure currencies that might not be traded at a physical exchange, ensuring a more level playing field against entities that might have the resources for premium advantages. The other bottleneck for low latency HFT can be resolved by offloading the work done by the software to the FPGA and maximizing its speed. This would be quite an arduous task that could include but not be limited to creating our own TCP/IP stack and WebSocket framing. If most of the above were to be implemented, our system could be competitive with the most cutting-edge systems.

IX. References

Setting up HPS Linux connection to FPGA:

[1] Setting up Linux on HPS:

https://people.ece.cornell.edu/land/courses/ece5760/DE1_SOC/DE1-SoC-UP-Linux/Linux.pdf

[2] Set Default gateway for internet access on Linux:

<https://askubuntu.com/questions/415023/connect-network-is-unreachable-ping>

(pseudo-IP route add default via <your gateway>(eg:192.168.136.1) dev eth0)

[3] Moving files from pc to HPS Linux (need internet):

<https://opensource.com/article/22/10/transfer-files-windows-linux-pscp>

[4] Setting up HPS-FPGA Connection with Platform Designer (Use BRAM Address):

<https://www.youtube.com/watch?v=-AHUBHDR4-s>

[5] Linux Subsystem for Platform Designer Usage:

<https://www.intel.com/content/www/us/en/docs/programmable/683472/21-4/installing-windows-subsystem-for-linux.html>

[6] BRAM Address: <https://www.cs.columbia.edu/~sedwards/classes/2023/4840-spring/designs/Sketchmaster.pdf>